For your goal—**backend engineering + AI/ML + top-company interviews**—I recommend practicing in this order:

Core Syntax
↓
Data Types
↓
Logic
↓
Loops
↓
Functions ← YOU ARE HERE
↓
Strings
↓
Collections
↓
Comprehensions
↓
Functional Programming
↓
Iterators & Generators
↓
Modules & Packages
↓
Files & JSON
↓
Exceptions
↓
OOP
↓
Testing
↓
Async Python
↓
Backend
↓
AI/ML

🧠 How to Practice Every Problem

For every problem, try this progression:

Level 1 — Solve normally

```
def factorial(n):
    ...
```

Level 2 — Handle invalid input

```
factorial(-5)
```

Level 3 — Convert into reusable functions

```
factorial(n)
```

Level 4 — Add type hints

```
def factorial(n: int) -> int:
    ...
```

Level 5 — Test edge cases

0
1
negative numbers
very large numbers
invalid input

Level 6 — Analyze

Ask:

- What is the time complexity?
- What is the space complexity?
- Can it be optimized?
- Is recursion appropriate?
- Is this memory efficient?

This is the mindset that separates **syntax learners** from **software engineers**.

PHASE 1 — CORE PYTHON FUNDAMENTALS

A. Variables, Input, Output, and Basic Operations

Beginner

1. Print your name, age, university, and department.
2. Take name and age as input.
3. Take two numbers and print their sum.
4. Calculate the area of a rectangle.
5. Calculate the area of a circle.
6. Convert Celsius to Fahrenheit.
7. Convert Fahrenheit to Celsius.
8. Convert kilometers to miles.
9. Convert seconds into hours, minutes, and seconds.
10. Calculate simple interest.
11. Calculate compound interest.
12. Calculate the average of three numbers.
13. Swap two variables.
14. Swap two variables without a third variable.
15. Calculate the total price after discount.
16. Calculate salary after tax.
17. Convert a number of days into years, months, and days.
18. Take multiple values in one line.
19. Parse `hh:mm:ss` input.
20. Build a basic calculator.

PHASE 2 — DATA TYPES

B. Numbers

21. Check whether a number is positive, negative, or zero.
22. Check even or odd.
23. Find the largest of two numbers.
24. Find the largest of three numbers.
25. Find the smallest of three numbers.
26. Find the absolute value.
27. Find quotient and remainder.
28. Count digits in a number.
29. Reverse a number.
30. Find the first digit.
31. Find the last digit.
32. Find the sum of digits.
33. Find the product of digits.
34. Find the average of digits.

35. Count even digits.
36. Count odd digits.
37. Find the largest digit.
38. Find the smallest digit.
39. Remove the last digit repeatedly.
40. Extract digits from right to left.
41. Check whether a number contains a particular digit.
42. Find the frequency of a digit.
43. Find the sum of even digits.
44. Find the sum of odd digits.
45. Check whether a number is divisible by 3, 5, or 7.

PHASE 3 — CONDITIONAL LOGIC

Your existing practice plan already includes problems such as leap year, grade calculation, BMI, login validation, ATM PIN, electricity bill, tax, discount, triangle type, and quadratic equations.

Add these:

46. Check leap year.
47. Classify age group.
48. Grade calculator.
49. BMI calculator.
50. Electricity bill calculator.
51. Income tax calculator.
52. Discount calculator.
53. ATM withdrawal validation.
54. Login validation.
55. Password strength validation.
56. Triangle validity.
57. Triangle type.
58. Quadratic equation solver.
59. Date validity checker.
60. Number range classifier.
61. Rock-paper-scissors decision logic.
62. Movie ticket price calculator.
63. Shipping fee calculator.
64. Electricity bill with slabs.
65. Mobile data package calculator.
66. Student scholarship eligibility.
67. Employee bonus eligibility.
68. Bank loan eligibility.
69. Multiple-condition access-control system.
70. Role-based permission checker.

PHASE 4 — LOOPS AND NUMBER LOGIC

Your uploaded plan includes factorial, Fibonacci, prime numbers, Armstrong numbers, perfect numbers, palindrome, reverse numbers, digit operations, multiplication tables, and patterns.

C. Essential Number Problems

71. Print 1 to n.
72. Print n to 1.
73. Print even numbers.
74. Print odd numbers.
75. Sum 1 to n.
76. Sum even numbers.
77. Sum odd numbers.
78. Multiplication table.
79. Factorial.
80. Factorial using recursion.
81. Fibonacci series.
82. Fibonacci using recursion.
83. Fibonacci using a generator.
84. Check prime.
85. Print primes in a range.
86. Count primes in a range.
87. Find the nth prime.
88. Find GCD.
89. Find LCM.
90. Check Armstrong number.
91. Print Armstrong numbers in a range.
92. Check perfect number.
93. Print perfect numbers in a range.
94. Check palindrome number.
95. Reverse a number.
96. Check strong number.
97. Check automorphic number.
98. Check Harshad number.
99. Check neon number.
100. Check spy number.
101. Check happy number.
102. Check duck number.
103. Check narcissistic number.

104. Count trailing zeros in `n!`.
105. Find the largest digit.
106. Find the second-largest digit.
107. Find the sum of factorials of digits.
108. Find the factorial of every list number.
109. Find the largest factorial from a list.
110. Generate all divisors.
111. Count divisors.
112. Sum divisors.
113. Check whether a number is abundant.
114. Check whether a number is deficient.
115. Find prime factors.
116. Count prime factors.
117. Find the largest prime factor.
118. Generate multiplication tables from 1 to 10.
119. Find numbers divisible by both `a` and `b`.
120. Find numbers divisible by neither `a` nor `b`.

PHASE 5 — FUNCTIONS ⭐⭐⭐⭐⭐

This is your current phase.

Your existing plan includes calculators, converters, recursion, GCD, LCM, default arguments, keyword arguments, `*args`, `**kwargs`, lambda, higher-order functions, decorators, closures, and generators.

I recommend doing these in order.

Level 1 — Function Fundamentals

121. Create a function that prints your name.
122. Create a function that returns your name.
123. Create a function that adds two numbers.
124. Create a function that subtracts two numbers.
125. Create a function that multiplies two numbers.
126. Create a function that divides two numbers.
127. Create a calculator using functions.
128. Create an even/odd function.
129. Create a positive/negative function.
130. Create a maximum function.
131. Create a minimum function.
132. Create a function to calculate the square.

133. Create a function to calculate the cube.
134. Create a function to calculate power.
135. Create a function to calculate factorial.
136. Create a function to calculate digit count.
137. Create a function to reverse a number.
138. Create a function to sum digits.
139. Create a function to check palindrome.
140. Create a function to check prime.

Level 2 — Parameters and Return Values

141. Function with one parameter.
142. Function with multiple parameters.
143. Function with default parameters.
144. Function with keyword arguments.
145. Function with positional arguments.
146. Function with mixed arguments.
147. Return multiple values.
148. Unpack multiple returned values.
149. Function returning a list.
150. Function returning a dictionary.
151. Function returning a tuple.
152. Function returning another function.
153. Function accepting a list.
154. Function accepting a dictionary.
155. Function accepting a function.

Level 3 — `*args` and `**kwargs`

156. Sum unlimited numbers using `*args`.
157. Find the maximum using `*args`.
158. Find the minimum using `*args`.
159. Calculate average using `*args`.
160. Build a calculator using `*args`.
161. Print user information using `**kwargs`.
162. Build a configuration function using `**kwargs`.
163. Merge dictionaries using `**kwargs`.
164. Create a function accepting both `*args` and `**kwargs`.
165. Build a logging function using `*args` and `**kwargs`.

Level 4 — Recursion

166. Recursive countdown.
167. Recursive count-up.
168. Recursive factorial.
169. Recursive Fibonacci.
170. Recursive sum from 1 to n.
171. Recursive sum of digits.
172. Recursive digit count.
173. Recursive reverse string.
174. Recursive palindrome check.
175. Recursive power.
176. Recursive GCD.
177. Recursive binary conversion.
178. Recursive decimal-to-binary conversion.
179. Recursive list traversal.
180. Recursive list sum.
181. Recursive maximum in a list.
182. Recursive minimum in a list.
183. Recursive linear search.
184. Recursive binary search.
185. Recursive tree-style nested dictionary traversal.

PHASE 6 — STRINGS ⭐⭐⭐⭐⭐

Your uploaded plan includes reverse string, palindrome, vowels, consonants, frequency, duplicates, spaces, capitalization, word count, compression, longest word, replacement, email validation, password validation, and Caesar cipher.

Practice:

186. Reverse a string.
187. Check palindrome.
188. Count characters.
189. Count vowels.
190. Count consonants.
191. Count digits.
192. Count spaces.
193. Count uppercase characters.
194. Count lowercase characters.
195. Find character frequency.
196. Find the most frequent character.
197. Find the first non-repeating character.
198. Remove duplicate characters.

199. Remove duplicate words.
200. Remove all spaces.
201. Replace multiple spaces.
202. Count words.
203. Find the longest word.
204. Find the shortest word.
205. Reverse each word.
206. Reverse word order.
207. Capitalize every word.
208. Convert snake_case to camelCase.
209. Convert camelCase to snake_case.
210. String compression.
211. Run-length encoding.
212. Caesar cipher.
213. Validate email.
214. Validate password.
215. Extract numbers from text.
216. Extract URLs.
217. Extract phone numbers.
218. Mask email addresses.
219. Mask phone numbers.
220. Build a text analyzer.

PHASE 7 — LISTS ⭐⭐⭐⭐⭐

Your existing plan includes maximum, minimum, second largest, reverse, rotate, merge, duplicate removal, frequency, sorting, searching, and matrix operations.

Practice:

221. Find maximum.
222. Find minimum.
223. Find second largest.
224. Find second smallest.
225. Reverse without `.reverse()`.
226. Rotate left.
227. Rotate right.
228. Remove duplicates.
229. Count frequencies.
230. Find duplicate values.
231. Find missing number.
232. Find common elements.
233. Merge two sorted lists.
234. Flatten nested lists.

235. Separate even and odd numbers.
236. Move zeros to the end.
237. Find pairs with a target sum.
238. Find triplets with a target sum.
239. Find the maximum subarray sum.
240. Find the intersection of two lists.
241. Find the union of two lists.
242. Implement linear search.
243. Implement binary search.
244. Implement bubble sort.
245. Implement selection sort.
246. Implement insertion sort.
247. Merge sort.
248. Quick sort.
249. Matrix addition.
250. Matrix subtraction.
251. Matrix multiplication.
252. Matrix transpose.
253. Spiral matrix traversal.
254. Rotate a matrix.
255. Find the diagonal sum.

PHASE 8 — TUPLES

256. Tuple packing.
257. Tuple unpacking.
258. Swap variables using tuple unpacking.
259. Nested tuple traversal.
260. Count elements.
261. Find index.
262. Convert list to tuple.
263. Convert tuple to list.
264. Return multiple values from a function.
265. Store database records as tuples.
266. Sort tuples by second element.
267. Sort tuples by multiple fields.

PHASE 9 — SETS

268. Remove duplicates.
269. Union.

270. Intersection.
271. Difference.
272. Symmetric difference.
273. Subset checking.
274. Superset checking.
275. Disjoint checking.
276. Find common interests.
277. Find students taking both courses.
278. Find users who visited two pages.
279. Find unique words in text.
280. Find unique characters.
281. Set comprehension.
282. Compare two datasets.

PHASE 10 — DICTIONARIES

Your existing plan includes student databases, employee databases, phone books, inventory, nested dictionaries, counters, merging, sorting, renaming, and inversion.

Practice:

283. Create a student database.
284. Create an employee database.
285. Create a phone book.
286. Create an inventory system.
287. Count word frequency.
288. Count character frequency.
289. Find the most frequent word.
290. Merge dictionaries.
291. Sort by keys.
292. Sort by values.
293. Rename a key.
294. Invert a dictionary.
295. Group values by category.
296. Find duplicate values.
297. Convert two lists into a dictionary.
298. Convert a dictionary into lists.
299. Nested dictionary traversal.
300. Update nested values.
301. Delete nested values.
302. Search nested dictionaries.
303. Build a JSON-like database.

304. Build a user authentication dictionary.
305. Build a product catalog.
306. Build a shopping cart.
307. Build a student gradebook.
308. Build a department employee system.
309. Build a cache using a dictionary.

PHASE 11 — COMPREHENSIONS

310. List of squares.
311. Even numbers.
312. Odd numbers.
313. Conditional comprehension.
314. Nested list comprehension.
315. Flatten a matrix.
316. Dictionary comprehension.
317. Set comprehension.
318. Character frequency.
319. Word length dictionary.
320. Invert dictionary using comprehension.
321. Filter dictionary values.
322. Transform dictionary values.
323. Build a matrix using comprehension.

PHASE 12 — FUNCTIONAL PYTHON

Your uploaded plan includes `lambda`, `map`, `filter`, `reduce`, `sorted(key=...)`, `zip`, `enumerate`, `any`, and `all`.

Practice:

324. Square values using `map`.
325. Convert strings to integers.
326. Filter even numbers.
327. Filter positive numbers.
328. Use `reduce` for sum.
329. Use `reduce` for product.
330. Sort users by age.
331. Sort products by price.
332. Combine lists using `zip`.

333. Create a dictionary using `zip`.
334. Number items using `enumerate`.
335. Validate all values using `all`.
336. Check any matching value using `any`.
337. Build a data transformation pipeline.

PHASE 13 — ITERATORS AND GENERATORS

The uploaded plan includes custom iterators, generator functions, generator expressions, `yield`, `next()`, and `iter()`.

Practice:

338. Create a custom iterator from `1` to `n`.
339. Create an even-number iterator.
340. Create an odd-number iterator.
341. Create a countdown iterator.
342. Create a Fibonacci generator.
343. Create a prime-number generator.
344. Create a factorial generator.
345. Create a file-line generator.
346. Create a large-data generator.
347. Create a generator pipeline.
348. Use `yield from`.
349. Create an infinite generator.
350. Build a paginated data generator.

This is particularly important for backend systems and AI data pipelines.

PHASE 14 — MODULES AND STANDARD LIBRARY

Your current plan already includes `math`, `random`, `datetime`, `os`, `sys`, `collections`, `itertools`, and `functools`.

Practice:

351. Create your own calculator module.
352. Create a math utilities module.
353. Create a validation module.
354. Create a string utilities module.
355. Create a package with multiple modules.
356. Use `__name__ == "__main__"`.
357. Use `collections.Counter`.
358. Use `defaultdict`.
359. Use `deque`.
360. Use `namedtuple`.
361. Use `itertools`.
362. Use `functools`.
363. Build a reusable utilities package.

PHASE 15 — DATE, TIME, JSON, REGEX

Your uploaded plan includes date/time projects, JSON conversion and files, and regex extraction/validation/replacement/splitting.

Practice:

364. Age calculator.
365. Date difference calculator.
366. Countdown timer.
367. Calendar generator.
368. Stopwatch.
369. Convert Python dictionary to JSON.
370. Convert JSON to Python.
371. Read JSON file.
372. Write JSON file.
373. Update JSON database.
374. Build a JSON-based user database.
375. Extract emails.
376. Extract phone numbers.
377. Extract URLs.
378. Extract dates.
379. Extract prices.
380. Validate emails.
381. Validate passwords.
382. Validate IPv4.
383. Validate Bangladesh phone numbers.
384. Remove HTML tags.
385. Split CSV text.
386. Build a log analyzer.

PHASE 16 — EXCEPTION HANDLING

Your uploaded plan already includes the major built-in exceptions, custom exceptions, multiple exceptions, `else`, and `finally`.

Practice:

387. Safe division.
388. Safe integer conversion.
389. Safe list access.
390. Safe dictionary access.
391. Safe file opening.
392. Handle multiple exceptions.
393. Create custom exceptions.
394. Validate user input with exceptions.
395. Build a banking exception system.
396. Build an authentication exception system.
397. Use `raise`.
398. Use exception chaining.
399. Add logging to exceptions.

🔥 PHASE 17 — MINI PROJECTS

Your uploaded plan recommends projects such as calculators, management systems, trackers, games, booking systems, shopping carts, attendance systems, weather APIs, and JSON databases.

For your backend career, I would prioritize these:

Beginner

1. Calculator
2. Contact Book
3. To-Do List
4. Expense Tracker
5. Quiz App

Intermediate

6. Student Management System
7. Library Management System

8. Inventory Management System
9. Banking System
10. Employee Management System

Stronger

11. JSON-based Database
12. Authentication System
13. Expense API
14. Inventory API
15. Blog API
16. Task Management API
17. URL Shortener
18. File Upload Service
19. Authentication Service
20. AI-powered API

⭐ Your Current Priority: Functions

Since you are starting functions today, do **not** immediately jump to decorators and generators.

Use this progression:

Day 1:
Basic functions
Parameters
Return values

Day 2:
Default arguments
Keyword arguments
Multiple return values

Day 3:
*args
**kwargs

Day 4:
Recursion

Day 5:
Functions with lists/dictionaries

Day 6:

Higher-order functions

Day 7:
Lambda + map/filter/reduce

Day 8:
Closures

Day 9:
Decorators

Day 10:
Generators

For each topic:

Learn
↓
Solve 10–20 problems
↓
Build one mini-project
↓
Refactor
↓
Add error handling
↓
Add tests

My Most Important Recommendation

Do **not** try to solve all 400+ problems mechanically.

Use **three levels**:

Level 1 — Fundamentals

Solve the problem normally.

Level 2 — Engineering

Convert it into:

```
def function_name(...) -> ...:
    ...
```

Add:

- Input validation
- Type hints
- Error handling
- Reusable design

Level 3 — Interview/Production

Ask:

Can I optimize it?
What is the time complexity?
What is the space complexity?
What happens with huge input?
Can I stream the data?
Can I test it?
Can I reuse it in an API?

For example, your **factorial problem** can evolve like this:

Factorial
↓
Loop implementation
↓
Recursive implementation
↓
Generator implementation
↓
Input validation
↓
Type hints
↓
Unit tests
↓
API endpoint
↓
AI/data-processing service

That is the kind of progression I recommend for you.

Your uploaded plan is a strong starting foundation, but for **top-level backend and AI/ML preparation**, your real target should be:

Master Python syntax → understand Python behavior and internals → solve problems → build software → test it → expose it through APIs → connect databases → optimize it.